

Report

Task 1: Build a “Tamper-Evident Logging System”

Chapter 1

Introduction

In modern digital systems, logs are one of the most critical sources of operational and security information. Every meaningful activity performed inside a system—such as user authentication, file access, system errors, configuration changes, or financial transactions—is typically recorded in the form of logs. These logs are used by system administrators, security analysts, and auditors to monitor system behavior, detect suspicious activity, troubleshoot failures, and investigate incidents.

In cybersecurity, logs play an especially important role because they act as a historical record of events. During a security incident, logs are often the first source examined to determine what happened, when it happened, and who was involved. For example, if a server is compromised, logs may reveal the source of the intrusion, the sequence of commands executed, and the systems that were affected. In enterprise environments, logs are also used to maintain accountability, support compliance requirements, and provide evidence during forensic investigations.

Despite their importance, conventional log files suffer from a major security weakness: they can be modified after they are created. Traditional logging systems usually store events in plain text files, flat files, or basic structured formats such as CSV or JSON. While these methods are simple and efficient for storing events, they do not guarantee that the recorded data remains unchanged. If an attacker gains access to the log file, they may alter or remove entries to hide malicious actions. Even legitimate insiders with sufficient access privileges can intentionally or accidentally modify logs, reducing their reliability.

This creates a significant challenge in security-sensitive systems. If logs can be changed without detection, they lose their value as trustworthy records. A compromised log file can no

longer be relied upon for audit, investigation, or evidence. Therefore, simply storing events is not sufficient; a secure logging system must also protect the integrity of stored log records.

To address this problem, this project implements a Tamper-Evident Logging System. The system is designed to ensure that once a log entry is created, any unauthorized attempt to modify, delete, or rearrange it can be detected. Rather than preventing access to the log file, the system focuses on making unauthorized changes visible.

This is achieved using cryptographic hash chaining. Each log entry stores its own cryptographic hash and also contains the hash of the previous log entry. This creates a linked chain of records where each entry depends on the one before it. Because of this dependency, changing any record disrupts the integrity of the chain and makes tampering detectable during verification.

The proposed system simulates the behavior of secure audit logging systems used in real-world environments such as banking systems, enterprise servers, digital forensic platforms, and compliance monitoring tools. Its purpose is to demonstrate how cryptographic integrity mechanisms can be used to improve trust, accountability, and reliability in system logging.

Chapter 2

Problem Statement and Objectives

2.1 Problem Statement

Traditional log systems are designed primarily to record events, but they do not provide sufficient protection against post-event manipulation. In most systems, logs are simply written to files in sequence and stored for later review. While this allows systems to maintain a record of operations, it does not ensure that those records remain trustworthy.

This weakness creates several major security concerns.

The first issue is unauthorized modification. If an attacker gains access to stored logs, they may alter the contents of existing entries to conceal malicious actions. For example, a failed login attempt could be changed into a successful one, or a record of unauthorized access could be rewritten to appear harmless.

The second issue is deletion of evidence. An attacker may remove specific log entries to erase traces of suspicious behavior. This is especially dangerous in systems that rely on logs for incident response or audit purposes.

The third issue is manipulation of event sequence. Even if the content of log entries remains unchanged, changing their order can distort the timeline of system events. This can mislead investigators and make forensic analysis unreliable.

These weaknesses reduce the value of logs in systems where accountability and trust are essential. In environments such as financial systems, enterprise security infrastructure, and digital forensics, logs must not only record activity but also provide assurance that the records have not been tampered with.

Therefore, a more secure logging mechanism is required—one that allows event recording while ensuring that any modification, deletion, or reordering of entries becomes detectable.

2.2 Objectives

The objective of this project is to design and implement a secure tamper-evident logging system that preserves the integrity of stored log records and detects unauthorized modifications.

The specific objectives are:

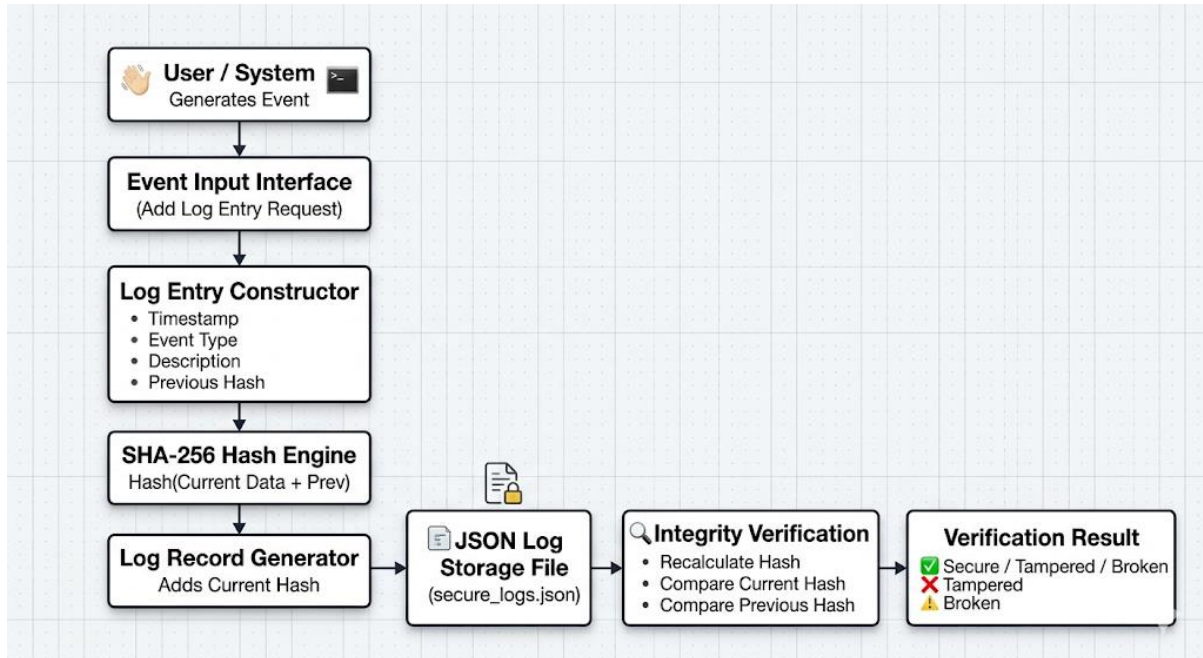
- To design a structured logging system for recording security-relevant events
- To create a cryptographic linkage between log entries using SHA-256 hashing
- To ensure that each log entry is dependent on the previous log entry
- To detect unauthorized modification of existing logs
- To detect deletion of log entries
- To detect reordering of log entries
- To implement an integrity verification mechanism for validating the complete log chain
- To simulate the behavior of secure audit logging systems used in real-world environments

The overall goal is to improve trustworthiness and accountability in digital logging systems by making unauthorized changes visible and traceable.

Chapter 3

Design of the Project

3.1 System Design Architecture Diagram



The system design architecture of the Tamper-Evident Logging System illustrates the complete lifecycle of a log event, beginning from event generation and ending with integrity verification. The architecture is designed in a sequential and layered manner so that every log passes through a controlled process before being stored and validated. Each component in the architecture performs a specific role in ensuring that the generated logs remain structured, traceable, and tamper-evident.

The system follows a linear trust pipeline in which every stage contributes to preserving the integrity of the final stored record. The flow begins when an event is generated and ends when the stored log chain is verified for authenticity and consistency.

1. User / System Generates Event

The first component in the architecture is the event source. This represents the origin of the activity that needs to be recorded in the logging system.

An event may be generated either by:

- a user action (such as login, logout, file access, failed authentication), or
- a system action (such as startup, shutdown, service failure, configuration update).

This component acts as the trigger point of the entire logging process. Every time a meaningful action occurs in the system, it is treated as an event that must be recorded.

The purpose of this stage is to capture raw activity that is relevant for monitoring, auditing, or forensic analysis.

Example events include:

- User login attempt
- File modified
- Unauthorized access detected
- Configuration changed
- Service restarted

This stage ensures that the logging process begins only when a relevant event occurs.

2. Event Input Interface

Once an event is generated, it is passed to the Event Input Interface.

This component acts as the entry point to the logging system. Its role is to receive the incoming event and forward it for structured processing.

In this project, the Event Input Interface is implemented through the command-line input mechanism, where the user manually enters:

- event type
- event description

This interface serves as the communication layer between the event source and the internal logging engine.

Its primary responsibilities are:

- accepting event input,
- validating that the event should be recorded,
- forwarding the event to the log creation module.

In practical systems, this component may be implemented as:

- command-line interface,
- application API,
- monitoring agent,
- system daemon,
- event collector.

In this project, it serves as the controlled gateway through which all log entries enter the system.

3. Log Entry Constructor

The Log Entry Constructor is the component responsible for transforming raw input into a structured log record.

This is one of the most important stages in the architecture because it standardizes the incoming event into a consistent format before it is hashed and stored.

At this stage, the system constructs a log entry using the following fields:

- **Timestamp** – records the exact date and time when the event occurred
- **Event Type** – specifies the category of the event (e.g., LOGIN, FILE_ACCESS, ERROR)
- **Description** – provides a human-readable explanation of the event
- **Previous Hash** – stores the cryptographic hash of the previous log entry

The Log Entry Constructor ensures that every record follows a fixed structure. This improves readability, consistency, and reliability during storage and verification.

The inclusion of the Previous Hash is especially important because this is the field that links the current log to the previous log in the chain.

Without this field, logs would remain independent records. With this field, they become cryptographically linked.

Thus, this module converts plain events into chain-aware log entries.

4. SHA-256 Hash Engine

After the log entry is structured, it is passed to the SHA-256 Hash Engine.

This component is responsible for generating the cryptographic fingerprint of the log entry.

The SHA-256 algorithm takes the following fields as input:

- Timestamp
- Event Type
- Description
- Previous Hash

It then computes a unique 256-bit hash value for the complete record.

This hash acts as the integrity signature of the log entry.

The purpose of this stage is to ensure that even the smallest change in the log data produces a completely different hash output.

For example:

- changing one letter in the description,
- modifying the event type,
- altering the timestamp,
- changing the previous hash

will generate a different hash value.

This property is what makes tampering detectable.

SHA-256 is used because it is:

- deterministic,
- collision-resistant,
- efficient,
- widely trusted in security systems.

This stage forms the cryptographic core of the entire architecture.

5. Log Record Generator

Once the hash is generated, the Log Record Generator finalizes the log entry.

This component takes:

- the structured log data from the Log Entry Constructor
- the generated hash from the SHA-256 engine

and combines them into the final log record.

At this point, the log entry becomes complete and contains:

- Timestamp
- Event Type
- Description
- Previous Hash
- Current Hash

The Current Hash is now added permanently to the log record.

This step transforms the temporary structured event into a complete tamper-evident log object that is ready for storage.

This module is important because it creates the final record format that will be written to persistent storage.

6. JSON Log Storage File (`secure_logs.json`)

After the log record is finalized, it is stored in the JSON Log Storage File.

This is the persistent storage layer of the architecture.

In this project, logs are stored in a file named:

```
secure_logs.json
```

This file acts as the central log repository and stores all log entries in chronological order.

JSON was selected because it is:

- lightweight,
- structured,
- human-readable,
- easy to parse,
- easy to verify.

Each log entry is stored as a JSON object, and all entries are stored in sequence as a list.

This stage ensures persistence, meaning the logs remain available even after the program terminates.

The storage file becomes the audit trail of the system.

7. Integrity Verification Module

The Integrity Verification module is responsible for validating whether the stored logs have remained unchanged.

This is the most security-critical stage of the architecture because it determines whether the log history is trustworthy.

The verification process performs three checks for every log entry:

1. Recalculate Hash

The system recalculates the hash of the current log entry using its stored contents.

2. Compare Current Hash

The recalculated hash is compared with the stored hash value.

If both values do not match, the log entry has been modified.

3. Compare Previous Hash

The system checks whether the current log's Previous Hash matches the actual hash of the previous log entry.

If it does not match, the chain has been broken.

This module detects:

- content modification,
- log deletion,
- log reordering,
- chain corruption.

This stage is responsible for identifying tampering.

8. Verification Result

The final stage of the architecture is the Verification Result module.

This component produces the final integrity status of the logging system based on the verification outcome.

The result may be one of the following:

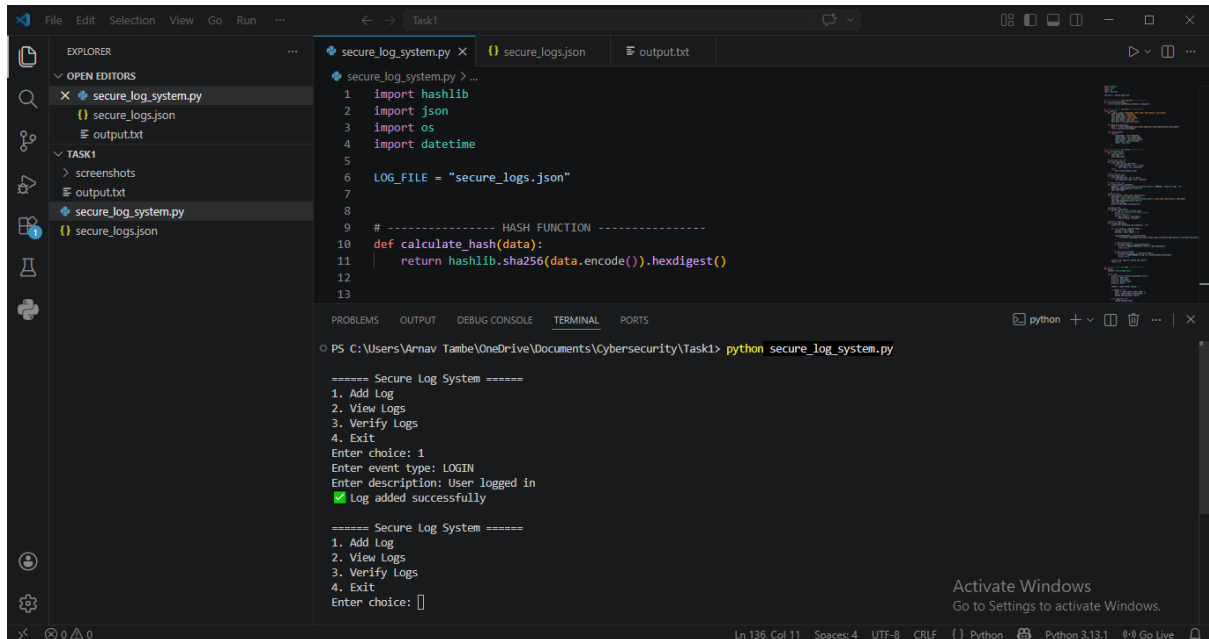
- **Secure** – all log entries are valid and the chain is intact
- **Tampered** – one or more log entries were modified
- **Broken** – the log chain is incomplete, reordered, or corrupted

This output gives the administrator or investigator a clear indication of whether the stored logs can be trusted.

It acts as the final decision layer of the architecture.

Chapter 4

Implementation



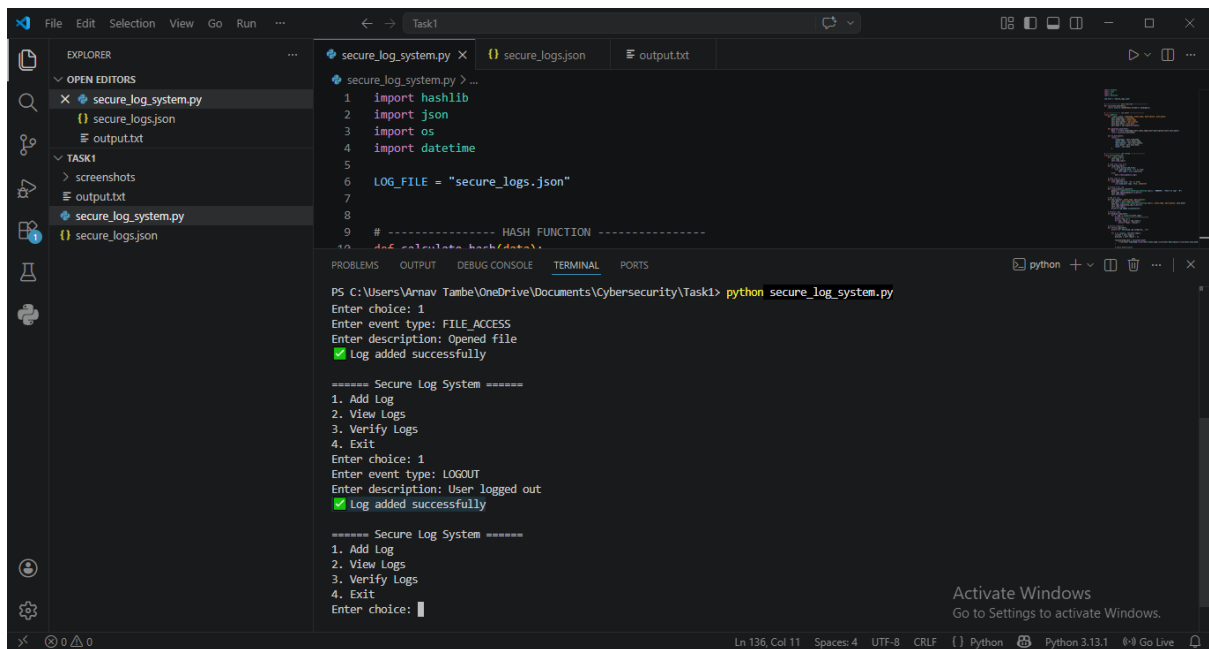
```
File Edit Selection View Go Run ...
Task1

EXPLORER
  OPEN EDITORS
    secure_log_system.py
    secure_logs.json
    output.txt
  TASK1
    screenshots
    output.txt
    secure_log_system.py
    secure_logs.json

secure_log_system.py
1 import hashlib
2 import json
3 import os
4 import datetime
5
6 LOG_FILE = "secure_logs.json"
7
8
9 # ----- HASH FUNCTION -----
10 def calculate_hash(data):
11     return hashlib.sha256(data.encode()).hexdigest()
12
13
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
python + - [] [] ... | x
PS C:\Users\Arnav Tambe\OneDrive\Documents\Cybersecurity\Task1> python secure_log_system.py

===== Secure Log System =====
1. Add Log
2. View Logs
3. Verify Logs
4. Exit
Enter choice: 1
Enter event type: LOGIN
Enter description: User logged in
✔ Log added successfully

===== Secure Log System =====
1. Add Log
2. View Logs
3. Verify Logs
4. Exit
Enter choice: 
```



```
File Edit Selection View Go Run ...
Task1

EXPLORER
  OPEN EDITORS
    secure_log_system.py
    secure_logs.json
    output.txt
  TASK1
    screenshots
    output.txt
    secure_log_system.py
    secure_logs.json

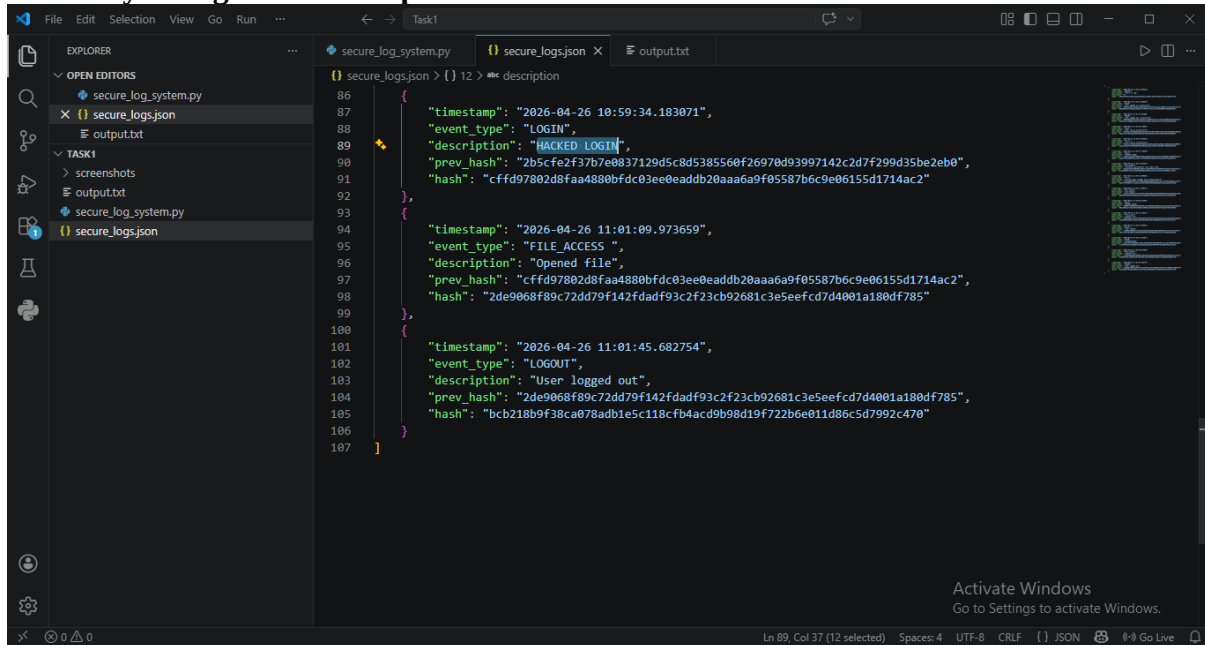
secure_log_system.py
1 import hashlib
2 import json
3 import os
4 import datetime
5
6 LOG_FILE = "secure_logs.json"
7
8
9 # ----- HASH FUNCTION -----
10 def calculate_hash(data):
11     return hashlib.sha256(data.encode()).hexdigest()
12
13
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
python + - [] [] ... | x
PS C:\Users\Arnav Tambe\OneDrive\Documents\Cybersecurity\Task1> python secure_log_system.py

Enter choice: 1
Enter event type: FILE_ACCESS
Enter description: Opened file
✔ Log added successfully

===== Secure Log System =====
1. Add Log
2. View Logs
3. Verify Logs
4. Exit
Enter choice: 1
Enter event type: LOGOUT
Enter description: User logged out
✔ Log added successfully

===== Secure Log System =====
1. Add Log
2. View Logs
3. Verify Logs
4. Exit
Enter choice: 
```

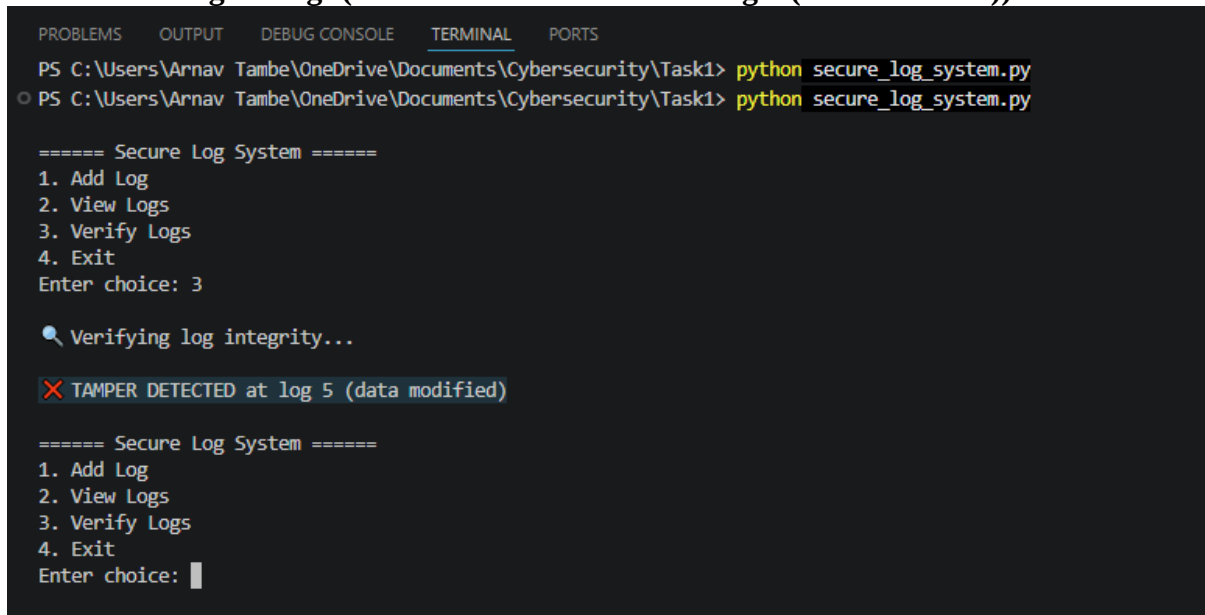
Manually changed the description:



The screenshot shows the Visual Studio Code editor with the file `secure_logs.json` open. The file contains a JSON array of log entries. The first entry has been manually modified to change the `description` from `"LOGIN"` to `"HACKED LOGIN"`. The `timestamp` is `"2026-04-26 10:59:34.183071"`, the `event_type` is `"LOGIN"`, the `prev_hash` is `"2b5cfe2f37b7e0837120d5c8d5385560f26970d93997142c2d7f299d35be2eb0"`, and the `hash` is `"cffd97802d8faa4880bfdc03ee0eaddb20aaa6a9f05587b6c9e06155d1714ac2"`. The second entry is for a `FILE_ACCESS` event with `description` `"Opened file"`. The third entry is for a `LOGOUT` event with `description` `"User logged out"`.

```
1  {  
2    "timestamp": "2026-04-26 10:59:34.183071",  
3    "event_type": "LOGIN",  
4    "description": "HACKED LOGIN",  
5    "prev_hash": "2b5cfe2f37b7e0837120d5c8d5385560f26970d93997142c2d7f299d35be2eb0",  
6    "hash": "cffd97802d8faa4880bfdc03ee0eaddb20aaa6a9f05587b6c9e06155d1714ac2"  
7  },  
8  {  
9    "timestamp": "2026-04-26 11:01:09.973659",  
10   "event_type": "FILE_ACCESS",  
11   "description": "Opened file",  
12   "prev_hash": "cffd97802d8faa4880bfdc03ee0eaddb20aaa6a9f05587b6c9e06155d1714ac2",  
13   "hash": "2de9068f89c72dd79f142fdad93c2f23cb92681c3e5eefcd7d4001a180df785"  
14 },  
15 {  
16   "timestamp": "2026-04-26 11:01:45.682754",  
17   "event_type": "LOGOUT",  
18   "description": "User logged out",  
19   "prev_hash": "2de9068f89c72dd79f142fdad93c2f23cb92681c3e5eefcd7d4001a180df785",  
20   "hash": "bcb218b9f38ca078adb1e5c118cfb4acd9b98d19f7722b6e011d86c5d7992c470"  
21 }  
22 ]
```

Detected Change in log: (TAMPER DETECTED at log 5 (data modified))



The screenshot shows a terminal window with the following output:

```
PS C:\Users\Arnav Tambe\OneDrive\Documents\Cybersecurity\Task1> python secure_log_system.py  
PS C:\Users\Arnav Tambe\OneDrive\Documents\Cybersecurity\Task1> python secure_log_system.py  
  
===== Secure Log System =====  
1. Add Log  
2. View Logs  
3. Verify Logs  
4. Exit  
Enter choice: 3  
  
🔍 Verifying log integrity...  
  
❌ TAMPER DETECTED at log 5 (data modified)  
  
===== Secure Log System =====  
1. Add Log  
2. View Logs  
3. Verify Logs  
4. Exit  
Enter choice: █
```

Chapter 5

Conclusion and Future Enhancements

The Tamper-Evident Logging System successfully demonstrates how cryptographic techniques can be used to strengthen the integrity and reliability of digital log records. The system was designed to address a key weakness in conventional logging mechanisms—the inability to detect unauthorized modifications after logs are stored. By linking each log entry with the previous one using SHA-256 hash chaining, the system ensures that any attempt to alter, delete, or rearrange records becomes immediately detectable during verification.

The implemented solution provides a lightweight yet effective approach for building trustworthy audit trails. It improves the reliability of log data, supports accountability, and makes the logging process more suitable for security monitoring and forensic analysis. The project also demonstrates how a simple structured design can simulate the core behavior of secure audit logging systems used in enterprise and security-sensitive environments.

Overall, the system achieves its primary objective of creating a secure and verifiable logging mechanism where the integrity of recorded events can be continuously validated. This makes it a practical foundation for systems where trust in recorded activity is essential.

Future Enhancements

Although the current implementation successfully provides tamper-evident logging, several enhancements can further improve its security, scalability, and practical usability.

In future versions, the system can be extended with **digital signatures** to provide stronger authenticity and non-repudiation, ensuring that log entries are not only intact but also verifiably created by authorized entities. **Encryption of stored logs** can also be introduced to protect confidentiality in addition to integrity.

The current file-based storage can be upgraded to a **database-backed logging system** for improved scalability, faster querying, and better handling of large log volumes. Support for **role-based access control (RBAC)** can also be added to restrict who can create, view, or verify logs.

To improve operational usability, the system can be enhanced with a **web-based dashboard** for real-time log monitoring, filtering, and verification. In addition, **real-time alert generation** can be implemented to notify administrators immediately when tampering is detected.

Further enhancements may also include:

- integration with SIEM or centralized monitoring tools,
- automated backup and recovery mechanisms,
- timestamp authority integration for stronger forensic trust,
- cloud-based secure log archival.

With these improvements, the system can evolve from a prototype-level implementation into a more robust and enterprise-ready secure logging solution.